PASSWORD POLICY AUDITOR

Cybersecurity Portfolio Project Report

Project type	Local Python security utility
Primary focus	Password policy validation and weak-pattern detection
Technology	Python 3.13, regular expressions, unittest
Cost	£0 — Python standard library only
Validation	21 unit tests passed; compliant and non-compliant CLI scenarios verified

Prepared from implementation details, test output, and report output supplied in the conversation.

1. Executive Summary

The Password Policy Auditor is a local command-line tool written in Python. It evaluates a supplied synthetic test password against configurable policy requirements, identifies selected weak patterns, and produces a text audit report. It demonstrates authentication-security concepts, input validation, regular-expression use, automated testing, and privacy-conscious reporting.

The application was verified using compliant and non-compliant synthetic test cases. The generated reports matched the command-line results. The automated test suite completed with 21 tests passing and no failures.

Verified outcomes

Validation item	Result
Strong synthetic test	COMPLIANT — all five policy checks passed
Weak synthetic test	NON-COMPLIANT — three failed checks and one weak pattern
Automated tests	21 passed; 0 failures; 0 errors
Report generation	Both compliant and non-compliant reports created
Report privacy	Reports contain audit results, not password values

2. Objectives and Scope

The objectives were to build a no-cost local password checker; support configurable policy settings; detect selected weak patterns; reject invalid inputs and policy configurations; generate a report without writing the password value; and verify behavior with automated unit tests.

The tool is a learning and demonstration utility. It is not an enterprise identity platform, password manager, breach-password checker, or complete password-strength estimator.

3. Design and Implementation

PasswordPolicy configuration. A dataclass stores minimum length, required character classes, and the maximum repeated-character threshold. Configuration validation rejects invalid values, including non-positive integer thresholds and non-boolean requirement settings.

Audit engine. The audit function checks minimum length, uppercase, lowercase, digits, and special characters. It also checks selected weak patterns: excessive repeated characters, a small set of common weak words, and selected alphabetical, numeric, and keyboard sequences.

Command-line interface. The program prompts for a synthetic test password using getpass, so typed characters are not echoed in the terminal. It displays checks and weak patterns, reports the overall result, and saves an audit report.

Report generator. The report includes the overall result, policy-check outcomes, failed requirements, weak patterns, and a security note. The password value is not included in the report.

4. Project Structure

Password-Policy-Auditor/
■■■ src/ (main.py, password_auditor.py, report_generator.py)
■■■ tests/ (test_password_auditor.py, test_report_generator.py)
■■■ reports/ (generated locally; excluded from Git)
■■■ evidence/
■■■ .gitignore
■■■ README.md

5. Security and Privacy Considerations

Only synthetic test passwords should be used. getpass suppresses terminal echo. Generated reports contain policy outcomes and detected patterns rather than the password value. The project .gitignore excludes generated report text files, virtual-environment files, Python cache files, and common local development artifacts.

Important limitation: getpass hides input on screen, but the password still exists temporarily in application memory while processed. Setting a variable to None does not guarantee secure memory erasure. Do not use this tool to process real user credentials.

6. Testing Methodology and Results

Automated tests were run with Python's built-in unittest framework using: `py -m unittest discover -s tests -v`. The supplied terminal output showed 21 tests completed in 0.110 seconds, followed by OK.

Test area	Coverage
Core policy checks	Length, uppercase, lowercase, digit, special character
Weak-pattern detection	Repeated characters, selected sequences, keyboard patterns, common weak words
Input/configuration validation	Custom policy, invalid thresholds, non-boolean settings, non-string input
Report generation	File creation, compliance status, password value excluded

7. Functional Test Results

Compliant scenario: all five policy checks passed, no weak patterns were detected, and the overall result was COMPLIANT. Report created: `password_audit_20261108_142404.txt`.

Non-compliant scenario: minimum length, uppercase, and special-character checks failed. The common weak password pattern detector also flagged the test value. Overall result: NON-COMPLIANT. Report created: `password_audit_20261108_142656.txt`.

Both reports were inspected with PowerShell Get-Content. Their contents matched the CLI results and stated that no password values are stored in the report.

8. Limitations and Future Improvements

Weak-pattern checks use a limited predefined set of substring patterns. They do not provide comprehensive dictionary analysis, breach-password screening, entropy estimation, or full Unicode-aware analysis. Sequence matching detects only the sequences coded into the application. The tool evaluates one password at a time and does not integrate with an authentication service.

Possible future improvements include a larger curated weak-pattern list, configurable pattern lists, JSON/CSV reporting, stronger international character handling, and a graphical interface. Testing should continue to use synthetic values.

9. Conclusion

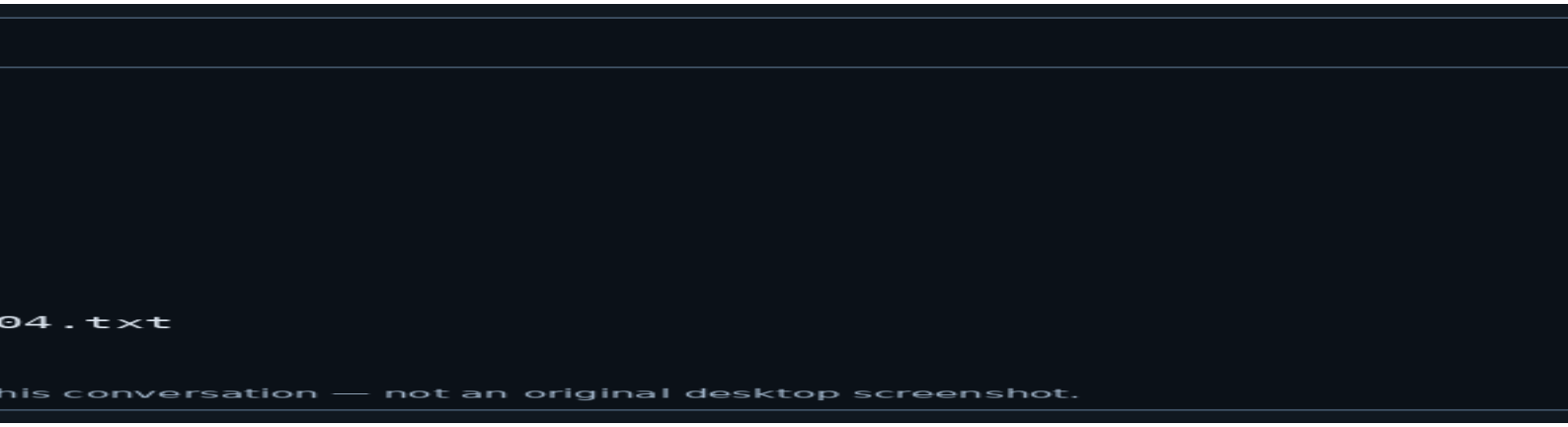
The project meets its stated portfolio scope: it performs configurable local password-policy checks, detects selected weak patterns, validates inputs and policy settings, generates reports, and has an automated test suite with 21 passing tests. The compliant and non-compliant CLI scenarios and their reports were verified from the supplied output.

The project demonstrates Python programming, regular expressions, defensive input validation, automated testing, and basic authentication-security awareness.

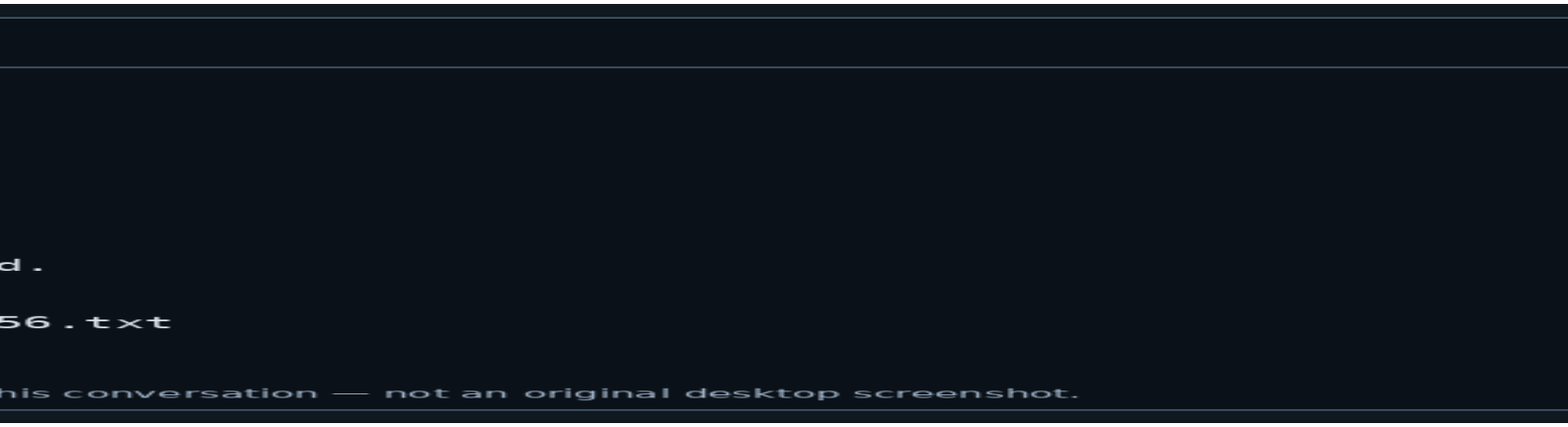
10. Evidence Appendix

These evidence panels reproduce the console and report results supplied in the conversation. They are formatted for this report and are not original desktop screenshots.

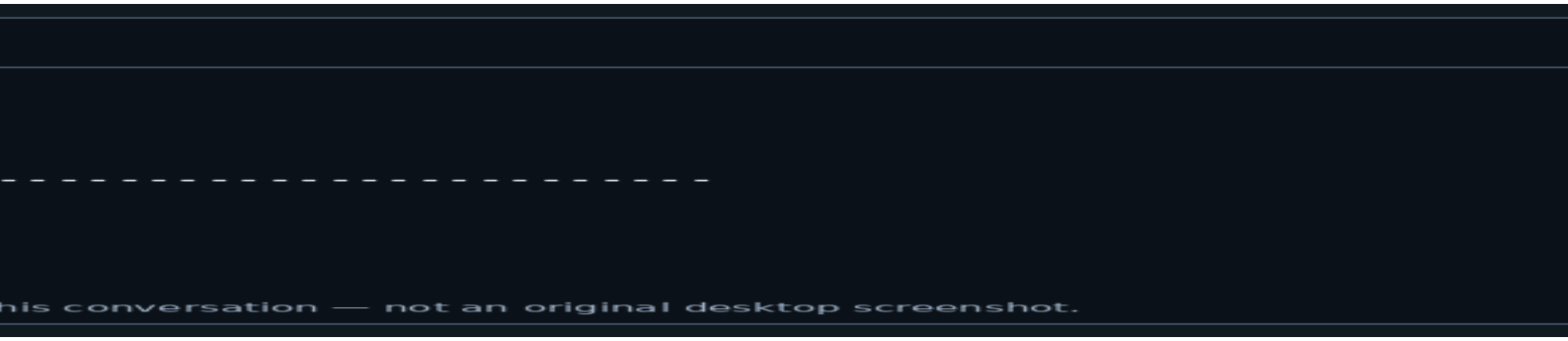
A. Compliant CLI execution



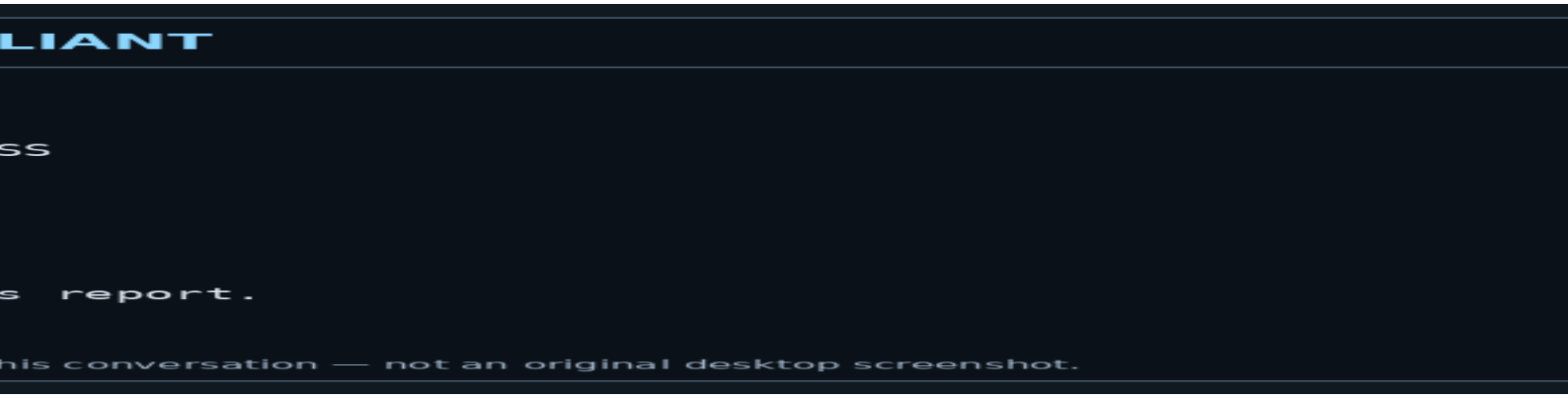
B. Non-compliant CLI execution



C. Automated unit-test run — 21 passed



D. Compliant generated report



E. Non-compliant generated report

